OWASP Top 10:2021

- Lecture By
  Pratidnya S. Hegde Patil
  Assistant Professor-IT Dept.

OWASP Top 10 - 2021

OWASP Top 10:2021: https://owasp.org/Top10/

TOP 10

A07:2021 Identification and Authentication Failures

(Broken Authentication)

Courtesy: https://portswigger.net/web-security/

Learning Path: Web Security Academy

Courtesy: https://u-next.com/blogs/cyber-security/broken-authentication/

Broken Authentication

- Broken Authentication is in one of the OWASP Top 10 Vulnerabilities. The essence of Broken Authentication is where you (Web Application) allow your users to get into your website by creating a new account and handling it for specific reasons. In Broken Authentication, whenever a user login into its account, a session id is being created, and that session id is allowed to that particular account only.
- Authentication ensures that only a verified user can access the information and privileges on the web application. It gets 'broken' when an attacker bypasses the process and impersonates the user on the application. These inherent weaknesses mentioned earlier can broadly be classified into two categories
  - poor session management and
  - poor credential management.

Session Management Flaws

- **Session Hijacking :**

  Without appropriate safeguards, web applications are vulnerable to session hijacking, in which attackers use stolen session IDs to impersonate users' identities. The most straightforward example of session hijacking is a user who forgets to log out of an application and then walks away from their device. A hacker can then continue their session.

- **Session ID URL Rewriting :**

  Another common avenue for session hijacking is "URL rewriting." In this scenario, an individual's session ID appears in the URL of a website. Anyone who can see it (such as via an unsecured Wi-Fi connection) can piggyback into the session. This is how Zoombombing happened.

Credential Management Flaws

- **Credential Stuffing:**

  When attackers access a database filled with unencrypted emails and passwords, they frequently sell or give away the list for other attackers to use. These attackers then use botnets for brute-force attacks that test credentials stolen from one site on different accounts. This tactic often works because people frequently use the same password across applications.

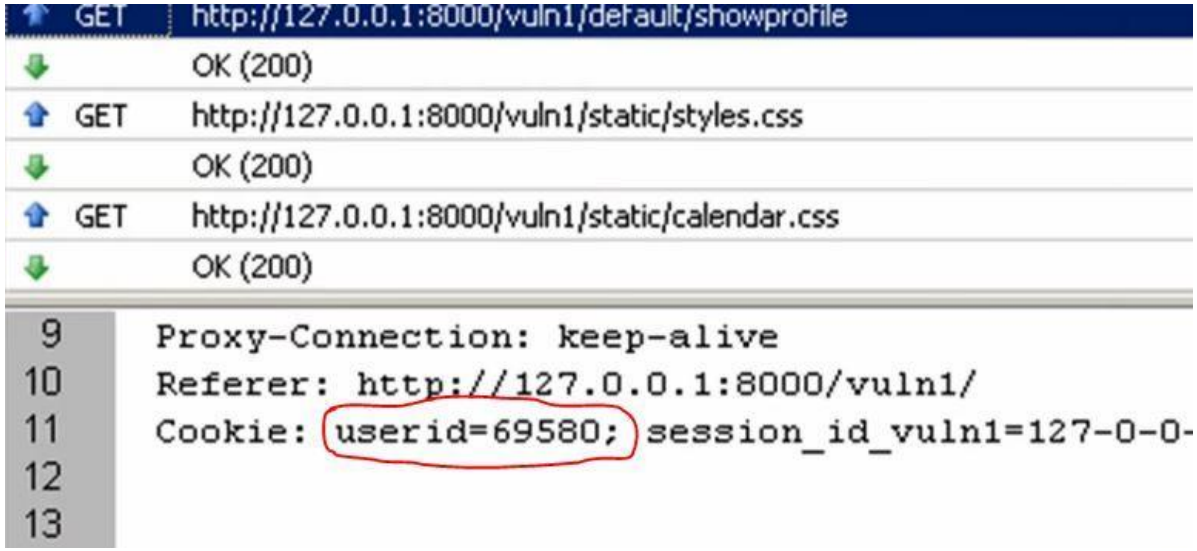
- **Password Spraying:** Password spraying refers to the use of the most common and weak passwords, such as ‘password’ or ‘123456’ by hackers trying to access secure accounts. Consequently, minimum password requirements have been introduced to avoid such attacks.
- **Phishing Attacks:** Hackers phish by sending users links to a website that resembles the original web application, to get users to divulge their login credentials. Phishing attacks can be easily prevented, however, with proper diligence and by verifying the web application in use.

Example: Broken Authentication Vulnerability Exploited (Exploiting the Cookie)

- Step 1 : Creating an account in a web application.

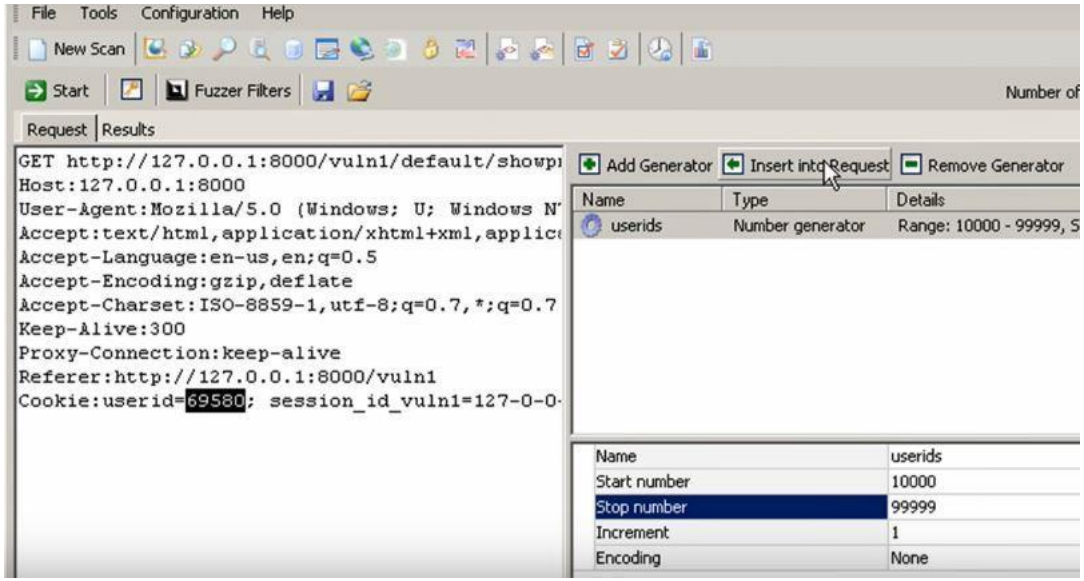
Example: Broken Authentication Vulnerability Exploited (Exploiting the Cookie)

- Step 2 : Intercept the Request with proxy tools such as Burpsuite and analyze the backend details. While intercepting the request, one will see something like this. Analyze the user id cookie it has generated for the specific user, i.e. you who have created an account.



Example: Broken Authentication Vulnerability Exploited (Exploiting the Cookie)

- Step 3 : Since the Cookie “UserId” has been sent to us by the server so it can be modified to check the profiles of other users by manipulating the cookie. We will try to brute force the USERID cookie and will check for the response.

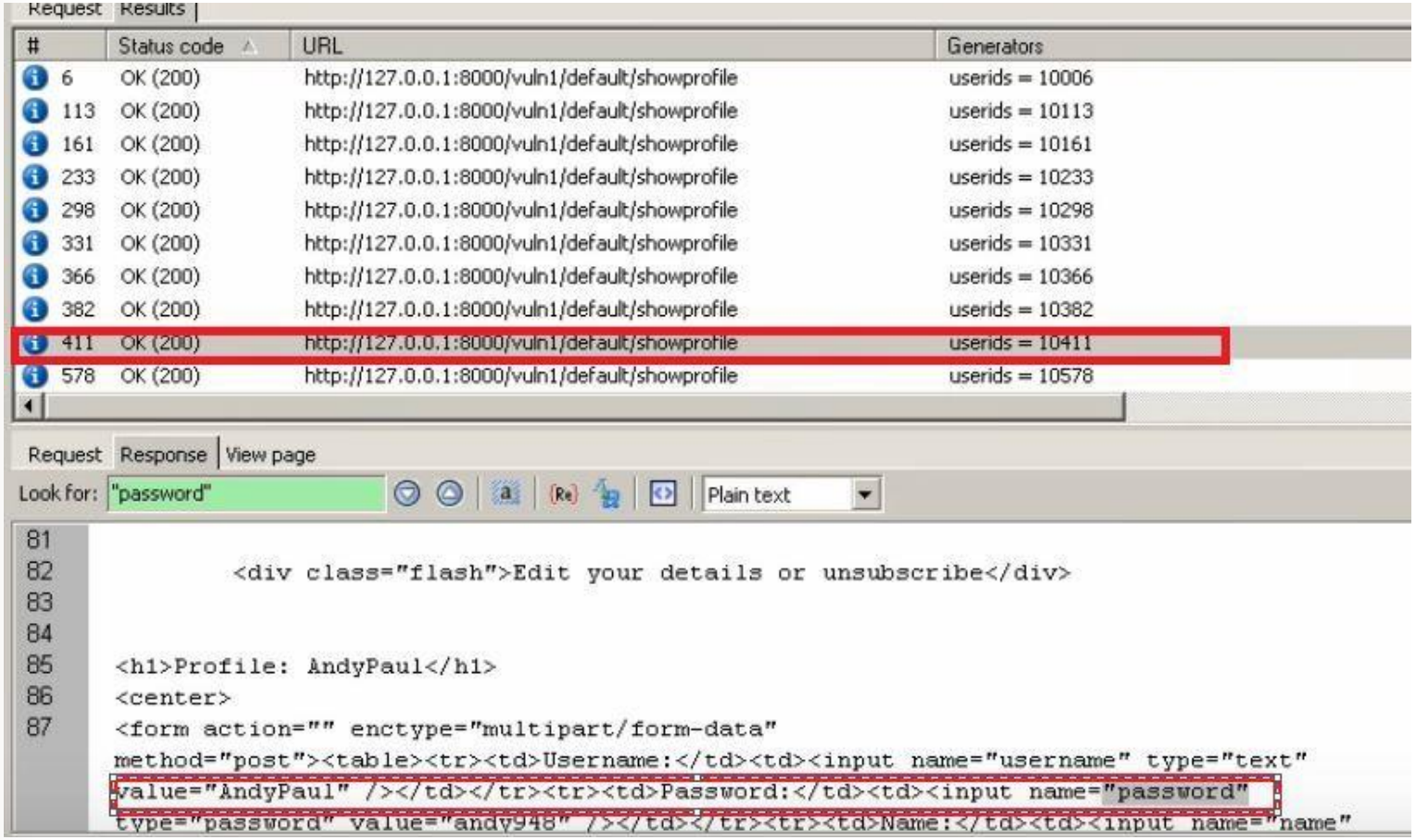


Example: Broken Authentication Vulnerability Exploited (Exploiting the Cookie)

- Step 4 : After the Bruteforcing the USERID cookie, we will see the response, which will be showing OK (200) code, it means that this particular combination work for the user id. Here in the image, we can see that there are so many requests that all are having OK status, and when we clicked on Request no 441, we saw that the user id brute force was 10411. Its username "usAndyPaul", and its default password was "PASSWORD". Hence in this way, we can extract an ample number of User Accounts if the Broken Authentication Vulnerability exists in the Web Application.

- [Image in next slide]

Example: Broken Authentication Vulnerability Exploited (Exploiting the Cookie)



Impacts of Broken Authentication Vulnerability

- Exposed numerous User Accounts
- Data Breaches
- Administrative Access
- Sensitive Data Exposure
- Identity Theft

Mitigation of Broken Authentication

- **Regulate session length:** The web application must be able to end web sessions after a period of inactivity that depends on the type of requirements of the user. A secure banking portal, for example, must automatically log out the user after a few minutes to avoid any risks of hijacked session IDs.

- **Improve session management:** The web application must be able to issue a new Session ID after every successful authentication. These IDs must be invalidated as soon as a session ends in order to prevent any misuse.

Mitigation of Broken Authentication

- Web URLs must be secure and must not include the Session ID in any form.
- **Multi-factor Authentication (MFA):** Among the OWASP top 10 A7:2021 broken authentication, the first tips is to implement Multi-factor Authentication to prevent attacks. MFA requires an additional credential to verify the user's identity. An example of MFA would be a One-Time Password (OTP) mailed or messaged to the user that allows for verification.
- **Disallow weak passwords:** Users must be required to set passwords of a specific length containing special characters, letters as well as numbers to prevent credential theft. Therefore, those passwords that do not meet the required complexity and length must be automatically rejected.

Mitigation of Broken Authentication

- **Breached password protection:** Employ a breached password protection mechanism that locks the accounts of users whose passwords have been compromised until they verify and change the password to a new one. This will ensure that if passwords are stolen, the organization is notified.
- **Strict credential recovery process:** The process to recover credentials must be strict, involving multiple verification checks to ensure that such recovery options are not misused by attackers.
- **Secure password storage:** Passwords must be encrypted, hashed, and salted as it helps slow down brute-force attacks or other attempts to infiltrate password databases.
- **Employ brute-force protection:** Applications should set a maximum limit for user-login attempts from a specific IP address, to prevent brute-force and credential stuffing attacks. Any user exceeding this limit must be disallowed from making any further attempts.

Single-factor and Multi-factor Authentication

- **Single-factor authentication:** So the first single authentication system that came out is the combination of the username and the password. The username determines the unique name of the user and the password is something private. This password is the only thing that will stop any unauthorized users to get access to the system.

- **Multi-factor authentication:** It requires more than one factor for successful authentication.

Single-factor authentication	Multi-factor authentication
There is a risk to security.	There is no risk of security.
There is a risk of a keylogger who can steal passwords and more. There is a chance of information getting stolen.	There is no risk of keylogger activity. There is no chance for the information to get stolen.
There is a chance of a phishing attack, where the attacker may deceive the user to enter the password or userid.	In multi-factor authentication, a phishing attack will not accomplish its purpose.
The user is not in full control.	The user is in full control.
Example of Single-factor authentication- **Steps:** 1.Password authentication by the online service entered by user.	Example of Multi-factor authentication- **Steps:** **1.Knowledge-** Entered Username and Password **2.Possession-** Entered Pin from mobile app **3.Inherence (Biometrics)-** Fingerprint verified

OAuth (Open Authorization)

Courtesy: https://portswigger.net/web-security/

Learning Path: Web Security Academy

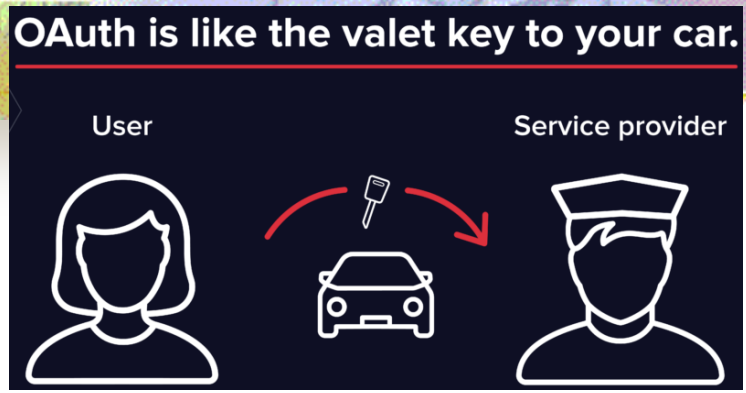
Courtesy: https://www.varonis.com/blog/what-is-oauth

OAuth

- OAuth is about authorization and not authentication.

- Authorization is asking for permission to do stuff. Authentication is about proving you are the correct person because you know things. OAuth doesn't pass authentication data between consumers and service providers – but instead acts as an authorization token of sorts.

Analogy



- OAuth is the valet key to your car. The valet key allows the valet to start and move the car but doesn't give them access to the trunk or the glove box.

- An OAuth token is like that valet key. As a user, you get to tell the consumers what they can use and what they can't use from each service provider. You can give each consumer a different valet key. They never have the full key or any of the private data that gives them access to the full key.

OAuth

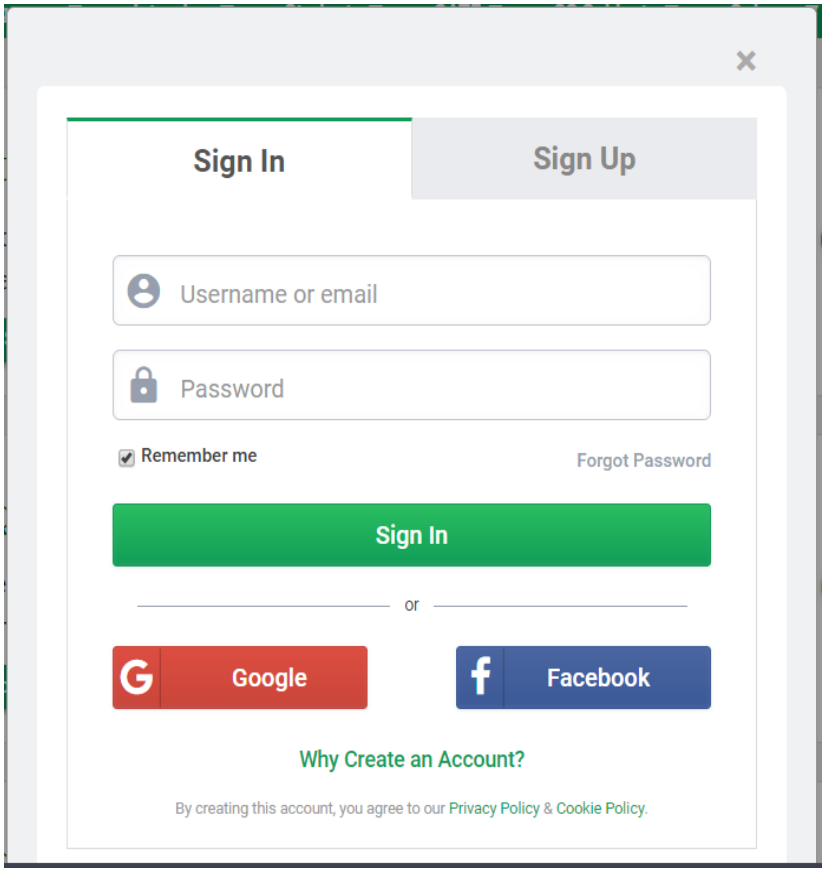
- OAuth is an open-standard authorization protocol or framework that provides applications the ability for “secure designated access.” For example, you can tell Facebook that it’s OK for ESPN.com to access your profile or post updates to your timeline without having to give ESPN your Facebook password. This minimizes risk in a major way: In the event ESPN suffers a breach, your Facebook password remains safe.

- OAuth doesn’t share password data but instead uses authorization tokens to prove an identity between consumers and service providers. OAuth is an authorization protocol that allows you to approve one application interacting with another on your behalf without giving away your password.

OAuth

- You might have seen a “login with Google” or “login with Facebook” button on the login/signup page of a website that makes easier to get using the service or website by simply logging into one of the services and grant the client application permission to access your data without giving Password. This is done with the OAuth.

- It is designed to work with HTTPS and it allows access tokens to be issued to the third party application by an authorization server with the approval from the owner.



Components in OAuth Mechanism

- **OAuth Provider :** This is the OAuth provider Eg. Google, FaceBook etc.
- **OAuth Client :** This is the website where we are sharing or authenticating the usage of our information. Eg. GeeksforGeeks etc.
- **Owner :** The user whose login authenticates sharing of information.

OAuth Examples

- The simplest example of OAuth is when you go to log onto a website and it offers one or more opportunities to log on using another website's/service's logon. You then click on the button linked to the other website, the other website authenticates you, and the website you were originally connecting to logs you on itself afterward using permission gained from the second website.

- A user sending cloud-stored files to another user via email, when the cloud storage and email systems are otherwise unrelated other than supporting the OAuth framework (e.g., Google Gmail and Microsoft OneDrive). When the end-user attaches the files to their email and browses to select the files to attach, OAuth could be used behind the scenes to allow the email system to seamlessly authenticate and browse to the protected files without requiring a second logon to the file storage system.

OAuth Examples

- Facebook apps are a good OAuth use case example. Say you're using an app on Facebook, and it asks you to share your profile and pictures. Facebook is, in this case, the service provider: it has your login data and your pictures. The app is the consumer, and as the user, you want to use the app to do something with your pictures. You specifically gave this app access to your pictures, which OAuth is managing in the background.

- Your smart home devices – toaster, thermostat, security system, etc. – probably use some kind of login data to sync with each other and allow you to administer them from a browser or client device. These devices use what OAuth calls *confidential authorization*. That means they hold onto the secret key information, so you don't have to log in over and over again.

OAuth Workflow

- **User :** A visitor who wants easy access over apps without the hassle of creating a new account.
- **Browser :** A web browser is a simple tool which is being used by the user to access the software application, here that is called client.
- **Client :** The client is a software application which can be a web app, desktop app, mobile phone app or a smart device.
- **Authorization Server :** Authorization Server authorizes user and client (software application) from OAuth API provider.
- **Resource Server :** When user and client (software application) are authenticated then the user can request for restricted resources through a client (software application) from OAuth API provider.

Example1: How OAuth Works?

1. User visits client (software application) and requests to log in through OAuth of let's say Facebook.
2. Client (software application) requests a browser for access.
3. Browser redirects access to Facebook's authorization server.
4. Authorization server asks the user to authenticate himself/herself.
5. The user enters his/her Facebook's login credentials and authorization server matches the credentials and on the successful match, it authenticates the user.
6. The authorization server then asks the user to authorize the client (software application).

Example1: How OAuth Works?

7. User authorizes client (software application) to get data from Facebook.
8. The authorization server redirects the user on the browser to the client (software application) with the authorization code.
9. Client (software application) uses authorization code with credentials (secret key) to request an Access token and refresh token.
10. The client (software application) then goes to the resource server and provides access tokens to get restricted resources from Facebook.
11. Now the user is successfully registered with the app and is logged in to the client (software application).

Example2: How OAuth Works?

- There are 3 main players in an OAuth transaction: the user, the consumer, and the service provider.
- In our example, Joe is the user, Bitly is the consumer, and Twitter is the service provider, who controls Joe's secure resource (his Twitter stream). Joe would like Bitly to be able to post shortened links to his stream. Here's how it works:
- **Step 1 – The User Shows Intent**
  - **Joe (User):** "Hey, Bitly, I would like you to be able to post links directly to my Twitter stream."
  - **Bitly (Consumer):** "Great! Let me go ask for permission."
- **Step 2 – The Consumer Gets Permission**
  - **Bitly:** "I have a user that would like me to post to his stream. Can I have a request token?"
  - **Twitter (Service Provider):** "Sure. Here's a token and a secret."
  - The secret is used to prevent request forgery. The consumer uses the secret to sign each request so that the service provider can verify it is actually coming from the consumer application.

Example2: How OAuth Works?

- **Step 3 – The User Is Redirected to the Service Provider**
  - **Bitly:** "OK, Joe. I'm sending you over to Twitter so you can approve. Take this token with you."
  - **Joe:** "OK!"
  - **- Bitly directs Joe to Twitter for authorization>**
  - This is the scary part. If Bitly were super-shady Evil Co. it could pop up a window that looked like Twitter but was really phishing for your username and password. Always be sure to verify that the URL you're directed to is actually the service provider (Twitter, in this case).
- **Step 4 – The User Gives Permission**
  - **Joe:** "Twitter, I'd like to authorize this request token that Bitly gave me."
  - **Twitter:** "OK, just to be sure, you want to authorize Bitly to do X, Y, and Z with your Twitter account?"
  - **Joe:** "Yes!"
  - **Twitter:** "OK, you can go back to Bitly and tell them they have permission to use their request token."
  - Twitter marks the request token as "good-to-go," so when the consumer requests access, it will be accepted (so long as it's signed using their shared secret).

Example2: How OAuth Works?

- **Step 5 – The Consumer Obtains an Access Token**
  - **Bitly:** "Twitter, can I exchange this request token for an access token?"
  - **Twitter:** "Sure. Here's your access token and secret."

- **Step 6 – The Consumer Accesses the Protected Resource**
  - **Bitly:** "I'd like to post this link to Joe's stream. Here's my access token!"
  - **Twitter:** "Done!"
  - In our scenario, Joe never had to share his Twitter credentials with Bitly. He simply delegated access using OAuth in a secure manner. At any time, Joe can login to Twitter and review the access he has granted and revoke tokens for specific applications without affecting others. OAuth also allows for granular permission levels. You can give Bitly the right to post to your Twitter account, but restrict LinkedIn to read-only access.

Example3: How OAuth Works?

- The authorization flow in a typical OAuth 2.0 implementation is a six-step process. In the example below, an online calendar creation application needs to be able to access a user's photos stored on their Google Drive:
  - **Step 1 :** The calendar creation application (the client) requests authorization to access protected resources, in this case image files, owned by the user (resource owner) by directing the user to the authorize endpoint.
  - **Step 2 :** The resource owner authenticates and authorizes the resource access request from the application, and the authorize endpoint returns an authorization grant to the client. The OAuth 2.0 protocol defines four types of grants: Authorization Code, Client Credentials, Device Code and Refresh Token.
  - **Step 3 :** The client then requests an access token from the authorization server by presenting the authorization grant returned from the authorize endpoint along with authentication of its own identity to the token endpoint. A token endpoint is a URL such as https://your_domain/oauth2/token.

Example3: How OAuth Works?

- **Step 4 :** If the client identity is authenticated and the authorization grant is valid, the authorization server or authentication provider -- Google's Authorization Server in this instance -- will issue an access token to the client.

- **Step 5:** The client can now request the protected resources from the resource server -- Google Drive in this example -- by presenting the access token for authentication.

- **Step 6 :** If the access token is valid, the resource server returns the requested resources to the calendar creation application (client).

- Now the calendar creation application can access and import the user's photos to create a calendar. Depending on the grant type issued in step two, the authorization flow may differ slightly. However, it still largely follows these core steps.

SAML vs OAuth

SAML	OAuth
SAML (Security Assertion Markup Language) is an alternative federated authentication standard that many enterprises use for Single-Sign On (SSO). SAML enables enterprises to monitor who has access to corporate resources.	OAuth is an open-standard authorization protocol (OAuth1.0) or framework (OAuth 2.0)that provides applications the ability for "secure designated access."
SAML uses XML to pass messages.	Uses JSON.
SAML drops a session cookie in a browser that allows a user to access certain web pages – great for short-lived work days, but not so great when have to log into your thermostat every day.	OAuth uses API calls extensively, which is why mobile applications, modern web applications, game consoles, and Internet of Things (IoT) devices find OAuth a better experience for the user.
SAML is geared towards enterprise security.	OAuth provides a simpler mobile experience.